Banya AI 2

Gradient-Free Self-Organization of Concept Embeddings by Frequency-Proportional Rumination

Hyukjin Han (bokkamsun@gmail.com) · ORCID 0009-0007-4132-1707

Banya Research Series, Part 2

Abstract

Rumination—reorganizing concepts using only internal embeddings, without any outside data—is a self-organization learning rule that never computes a gradient: concept embeddings (coordinates that encode words as numeric vectors) pull their neighbors together and cluster on their own, with order arising purely from internal interactions rather than external instruction. Seeds are drawn with probability proportional to frequency (how often a word occurs); the same-kind neighbors of each seed (cosine nearest neighbors, concepts close in meaning) are pulled toward their centroid (the mean position of the vectors); and at every step a decay (a restoring force toward the original position) pulls all concepts slightly back toward their original embeddings, so the two forces balance and the system settles into an equilibrium without any target value. The parts responsible for grammar (syllable embeddings, the output head, and operation-plane embeddings) are frozen, i.e., excluded from learning. This learning uses no backpropagation, no loss function, and no external training data: it edits the embedding array directly, using only the arithmetic of the forward pass (the forward computation from input to output). This paper measures the following under this rule. First, while same-kind cohesion rises from 0.127 to 0.167 (about 31 percent), the grammar metric—holdout, the next-character prediction difficulty on evaluation text unseen in training—is essentially preserved, moving from 5.006 to 5.087. Second, leaving the rule on for 2000 steps does not diverge but converges to an equilibrium (noise norm from 1.0 to 0.14). Downstream two-candidate discrimination (16-question 2AFC) is also maintained before and after rumination, with no flips even in a stress run that forcibly reorients the candidate word embeddings. Third, feedback rumination organizes similarity on the data plane but does not move the operation plane (the reasoning axis), pinpointing that reasoning is an axis distinct from similarity and requires a loop other than feedback. The conclusion is that rumination is responsible for organizing data similarity, while reasoning and its integration remain the task of a different mechanism. Where Part 1 was an engine that computes exact gradients without automatic differentiation, this paper is learning with no gradient at all.

Measured

The quantitative figures in this paper are values measured in the writing environment (see Appendix B). The frozen model that serves as the learning background and the evaluation sets are the ones already trained and used as-is. The theoretical background of rumination (a self-referential view of nature) is in a separate axiom document [1]; the claims of this paper are not deduced from those axioms but come from the measurements above.

Contents

Terminology Legend

Terms are defined in the tables below. One concept goes by one name only, and the text and figures of all seven parts follow these tables. The in-code column gives the identifier that implements the concept in the enclosed code, and the standard-term column gives the closest name in common use; cells are filled only where they differ.

Terminology legend 1. Engine and credit

Term	Meaning	In code	Standard term
exact adjoint	The backward computation derived by hand as closed forms for every operation; the proper name of this engine and its credit chain	`banya_core.py`	manual back-propagation
adjoint	The partner operation of the forward pass; it carries the gradient exactly backward	`bwd` kernels	adjoint
credit	The loss gradient at a hidden state; the responsibility a parameter bears for the outcome	delta (δ)	gradient
credit chain	The skeleton that walks the cache in reverse, applying each adjoint to carry credit back to the input	`backward`	back-propagation
adjoint agreement	The check that the adjoint formulas produce the same values as finite differences	per-part probes	gradient check
circulant-matrix channel mixing	Channel mixing with a circulant matrix, computed as componentwise products under rfft		circular convolution
relative-distance mixing	Mixing positions with shared coefficients that depend only on distance	`m_mat_w_lag`	Toeplitz mixing
temporal mixing	The umbrella name for any path by which other positions' information flows into the current one		token mixing
mixing heads	The number of parallel heads a mixing operation is split into		attention heads
filter ladder	The convolutional channel transform whose filter scale varies with depth, from a few wide filters to many narrow ones	`m_mat_w_filter`	convolutional filterbank
window	The token positions the model sees at once; as a measurement unit, a fixed-length text slice	`T`	context window
probe	The verification script that replays every measured figure together with its target value	`probe` folder	reproduction script
frozen model	A developmental-stage model whose training is stopped and fixed	`model` npz	frozen checkpoint
holdout	Evaluation text never used in training		held-out set

Terminology legend 2. Bi-token and the gate

Term	Meaning	In code	Standard term
bi-token	The structure that gives one token an orthogonal pair of embeddings, a data plane and an operation plane	USE_BITOKEN	
data plane	The embedding plane carrying a token's content; it enters at the add slot	m_mat_w_data_axis	token embedding
operation plane	The embedding plane carrying a token's operation instruction; it enters only at the multiply slot	m_mat_w_operate_axis	
add slot, multiply slot	The residual-addition position where the data plane lands and the gate-multiplication position where the operation plane acts		residual connection, gating
gate	The valve that sets each channel's pass-through between 0 and 1; the previous token's operation plane is injected into it	m_mat_w_gate	gating
one-step shift	The one-position displacement by which the previous token's operation plane is injected into the next position's gate		
operation-as-weight	The phenomenon in which operation-plane embeddings emerge as weights without any labels		
consistent operator	An operator that acts in a consistent direction on any data		
direction consistency	The mean cosine of the directions in which one operation bends different data; the quantitative sign of operator emergence	p_direction_consistency	
rotation-operator gate	The gate that reads the operation plane as Euler rotation angles; abbreviated in the text as the rotation gate	paper7_rotation.py	
primitive operation	An operation the gate can express directly in a single-hop instruction		primitive
depth-excluded discrimination	The deliberately biased measurement that restricts depth to one block so the gate is the only remaining variable	Part 7 probe	
self-feedback	The scoring mode that feeds the model's own output back as input and scores the final state		autoregressive rollout

Terminology legend 3. Rumination and development

Term	Meaning	In code	Standard term
Rumination	Self-organizing learning that pulls embeddings toward their neighbors without computing any gradient	Part 2 probes	self-organization
same-kind group	The set of close-in-meaning concepts gathered from a seed by nearest-cosine traversal		cluster
centroid pull, decay	The two forces of Rumination: pulling toward the mean and restoring toward the original; equilibrium emerges from their balance	`ALPHA`, decay rate	
World-first	The curriculum order that develops sensory, spatial, and logical axes before language	`banya_world_data`	curriculum learning
developmental stage	A checkpoint stage defined by step count and named after human growth		
axis	A property scale formed as a direction inside the embedding: sensory, ordinal, categorical		representation axis

Terminology legend 4. Vocabulary and metacognition

Term	Meaning	In code	Standard term
syllable atom	The syllable-level token forming the bottom layer of the vocabulary	`AtomTokenizer`	character-level token
bundle	A concept token made by binding syllable atoms into the dictionary; used only as a vocabulary unit		subword
cache-dictionary tokenization	The two-layer vocabulary system that puts a bundle dictionary on top of the atom layer	Part 5 probes	contrast with subword tokenization
certain, vague, unknown	The three knowing states separated by the model's own output distribution	pmax, entropy	uncertainty estimation
routing	Assigning an answer's processing path according to the state		routing
reinterpretation	The procedure that re-runs a vague hidden state together with its context to lift it to certain	Part 6 probes	
similarity consolidation	The axis along which concepts are organized by data-plane similarity		
inference traversal	The axis along which inference is carried forward on the operation plane		
self-utterance	The target state that weaves similarity consolidation and inference traversal so the model continues speech by itself		
Banya framework	The fifteen-axiom system forming the series' background; not the ground of the claims in the text		

Symbol Legend

We define the symbols that the equations and the text of this paper use together. Local symbols that appear in only one equation are listed in the symbol line below that equation.

Symbol	Name	Definition and meaning	Type
E	Data-plane embeddings	Embedding matrix holding the meaning (similarity) of concepts. The only object rumination edits: only the concept (bundle) columns are edited while the grammar parts stay frozen	Matrix
E_0	Original data-plane embeddings	Copy of E at the start of rumination. The reference the decay restores toward at every step	Matrix
G	Same-kind group	Set of concepts close in meaning, gathered from one seed by priority traversal. Upper bound 60, child width 3	Set
c	Centroid	Mean position of the same-kind group's embeddings; the target of the pull	Vector
α	Pull strength	Rate of pulling each embedding toward the centroid. Standard setting 0.05; the Section 5.2 grid uses 0.03, 0.06, and 0.1	Scalar
λ	Decay rate	Rate of restoring the whole concept layer toward the original E_0; the directional counterforce to the pull. Standard setting 0.012; the Section 5.2 grid uses 0.003 and 0.006	Scalar
ρ	Budget decay	Factor multiplied into the budget at each depth when expanding the same-kind group. Value 0.8	Scalar
τ	Leaf threshold	Boundary below which a branch stops: expansion ends when the best child's cosine similarity times the budget falls below it. Value 0.14	Scalar
w	Rumination probability	Seed selection probability proportional to word frequency; 16 seeds per step are drawn with it	Vector
d	Depth	Number of steps away from the seed in the same-kind traversal; the budget shrinks as ρ^d with depth	Integer
j	Concept index	Concept number indicating a candidate child (Eq. 3-1) or a member of the same-kind group (Eq. 3-2)	Integer
cos	Cosine similarity	Resemblance measured by the angle between two vectors (-1 to 1). Used for child selection, the leaf test, and cohesion	Function

1. Introduction

Standard neural-network training defines a loss and updates parameters with its gradient. Part 1 of this series [2] presented an engine that computes that gradient with hand-derived exact adjoints, without any automatic differentiation library. This paper addresses the opposite extreme: no gradient is computed at all. There is no loss, no backpropagation, and no external training data. Instead, already-learned concept embeddings are reorganized using only their geometry.

Concretely, frequently occurring concepts are ruminated in proportion to their frequency, and the embeddings of same-kind neighbors close in meaning to each concept are pulled toward a common centroid so that they cluster. With only a clustering force, all concepts would collapse to a single point, so a decay that pulls each embedding slightly back toward its original position is applied at every step, letting the two forces balance. The syllable embeddings, the output head, and the operation-plane embeddings—the parts responsible for grammar—are frozen to preserve the ability to speak. The process edits the embedding array directly using only forward-pass arithmetic (means, cosines, normalization) and performs no backward computation whatsoever.

Rumination differs from ordinary learning. With no loss, no backpropagation, and no outside data, it reorganizes concepts using only material already learned. The result is used in two places: fed back into the training loop, it becomes retraining without outside data; invoked during dialogue (inference), it becomes a skill for deep thinking.

1.1 Contributions

- Gradient-free self-organization rule : We formalize a learning rule consisting of frequency-proportional rumination, same-kind neighbor centroid pull, decay toward the original, renormalization, and grammar freezing (Section 3).
- Simultaneous compression and grammar preservation : We show by measurement that the grammar holdout is preserved while same-kind cohesion rises (Section 5).
- Equilibrium without a target value : We show by a long run that the balance of pull and decay alone converges to an equilibrium without any set point (Section 5).
- Honest negative results : We show that ruminating the data plane and the operation plane together yields no gain, and that internal rumination compresses but does not create (Section 6).

2. Related Work

Rumination overlaps in purpose with several lines of work while differing in means. Hebbian learning, which strengthens connections between co-activated neurons, is close in that it organizes representations with a local rule [3], but rumination moves embedding coordinates directly rather than connection weights. Knowledge distillation, which transfers a large model's knowledge to a smaller one [4], and born-again networks, which repeatedly train a model with itself as teacher [5], resemble it in reusing self output, but they remain gradient-based learning and target an external answer distribution. Rumination has neither a target distribution nor a gradient. Work on representation collapse [6] studies the phenomenon of representations collapsing to a single point; rumination structurally prevents this collapse by using decay as a directional restoring force.

The distinctions are threefold. First, there is no gradient and no backpropagation (forward-pass arithmetic only). Second, there is no external training data (only the geometry of already-known concepts is reorganized). Third, there is no target value (the system stops on its own at the equilibrium of pull and decay). The theoretical motivation from a self-referential view is in a separate axiom document [1], but the results of this paper come from measurements independent of it.

3. Notation and Learning Rule

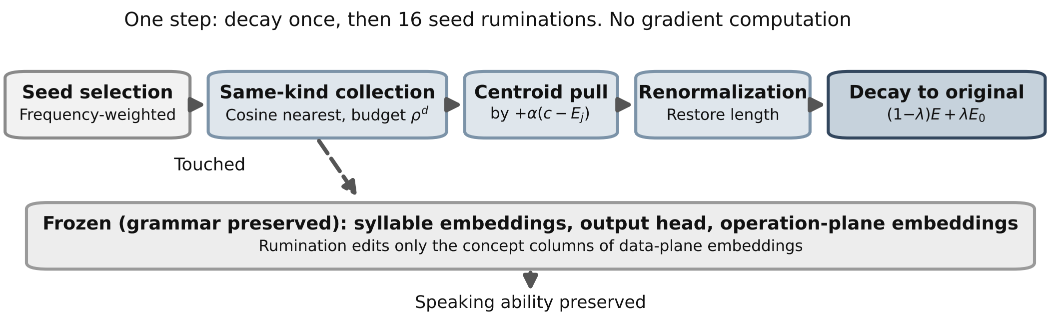
Table 3-1. Symbol legend

Symbol	Name	Definition and meaning	Type
E	Data-plane embeddings	Embedding matrix holding the meaning (similarity) of concepts. The object rumination edits	Matrix
E_0	Original data-plane embeddings	Copy of E at the start of rumination. The reference the decay restores toward	Matrix
G	Same-kind group	Set of concepts close in meaning, gathered from one seed	Set
c	Centroid	Mean position of the same-kind group's embeddings	Vector
α	Pull strength	Rate of pulling toward the centroid. Value 0.05	Scalar
λ	Decay rate	Rate of restoring toward the original. Value 0.012	Scalar
ρ	Budget decay	Budget multiplied at each depth when expanding the same-kind group. Value 0.8	Scalar
τ	Leaf threshold	Boundary below which no further expansion occurs. Value 0.14	Scalar
w	Rumination probability	Seed selection probability proportional to frequency	Vector

3.1 What Is Touched and What Is Frozen

Rumination edits only the concept (bundle) columns of the data-plane embeddings E. Three parts responsible for grammar are frozen: the syllable embeddings (the sound units of words), the output head (the layer that selects the next word), and the operation-plane embeddings (the axis responsible for transformations). This freezing is why the ability to speak does not collapse even as rumination organizes concepts (confirmed in Section 5).

Figure 3-1. Learning rule for one rumination step



One rumination step consists of one decay pass that pulls the entire concept layer slightly back toward the original, followed by 16 ruminations that pull seeds drawn in proportion to frequency toward their centroids. No stage uses a loss or a gradient. The three parts responsible for grammar (boxes at the bottom) are frozen, preserving the ability to speak.

3.2 Gathering the Same-Kind Group

Neighbors close in meaning to a seed concept are gathered by priority traversal. Children are cosine nearest neighbors (the concepts closest in meaning), and at each depth d the budget is multiplied by ρ, so expansion weakens with distance.

Eq. 3-1. Budget and leaf test of the same-kind group

$$b_d = \rho^d, \qquad \text{leaf} \iff \max_j \cos(\text{node}, j) \cdot b_d < \tau$$

Formula

Symbols: b_d is the budget at depth d; ρ is the budget decay (0.8); cos is cosine similarity (resemblance measured by the angle between two vectors, from -1 to 1); τ is the leaf threshold (0.14); a leaf is an endpoint that expands no further.
How it works: the budget shrinks geometrically with depth, so concepts far from the seed are naturally cut off. When the best child's similarity times the budget falls below the threshold, that branch stops.
In practice: for example, if the neighbors of the seed "bear" are "animal, puppy, rabbit," the budget at depth 1 is $0.8^1 = 0.8$, and similarity 0.3 times 0.8 gives 0.24, which exceeds the threshold 0.14, so expansion continues. Grammar fragments far from the seed have low similarity and are not selected within the budget, so the traversal stays on topic.

3.3 Pull and Decay

Eq. 3-2. Centroid pull and decay toward the original

$$c = \frac{1}{|G|}\sum_{j \in G} E_j, \qquad E_j := \big(E_j + \alpha\,(c - E_j)\big) \cdot \frac{\|E_j^0\|}{\|E_j + \alpha(c - E_j)\|}, \qquad E := (1-\lambda)\,E + \lambda\,E_0$$

Formula

Symbols: G is the same-kind group; c is its centroid; α is the pull strength (0.05); $\|E_j^0\|$ is the original length of that embedding; λ is the decay rate (0.012); E_0 is the original embeddings.

How it works: the middle equation pulls each embedding toward the centroid by α and then restores its original length. Only directions are gathered; magnitudes are preserved. The right equation is the decay that restores the entire concept layer toward the original by λ, which acts as the directional counterforce to the pull.
In practice: the pull gathers frequently ruminated concepts tightly together, while the decay pulls every concept slightly back toward its original position. For frequently ruminated concepts the pull beats the decay and they tighten; for rare concepts the decay wins and they stay as they were. Cohesion stops on its own where the two forces tie (Figure 5-3).

A small example shows why the decay is a directional counterforce. Suppose one embedding starts at the original $E_0 = (1, 0)$ and moves toward the centroid under the pull. The decay restores it toward the original at every step.

Step	**Position after pull**	**Position after decay** $(1-\lambda)x + \lambda E_0$
1	$(0.90,\ 0.30)$	$(0.9012,\ 0.2964)$
2	$(0.82,\ 0.50)$	$(0.8224,\ 0.4940)$

The stronger the pull, the closer the embedding moves to the centroid, but the decay drags it back toward the original a tiny bit each time. The point where the pull and the restoration tie is the equilibrium; without this counterforce, all concepts would collapse to a single point.

4. Experimental Setup

The learning background is a single frozen model. An elementary-stage model (about 190,000 steps) trained with the developmental curriculum (covered in Part 4) is frozen, and rumination is applied only to its data-plane embeddings. The live training model is not touched. There are five measurement metrics.

Table 4-1. Measurement metrics

Metric	Meaning	Direction
Holdout bits/char	Average information content when predicting the next character on evaluation text unseen in training. A measure of whether grammar is intact	Lower is better
Same-kind cohesion	Mean pairwise cosine of embeddings within the same concept group. Degree of clustering	Higher is tighter
Cross-kind separation	Cosine between different concept groups. Degree of non-mixing	Lower is more separated
Core strength	Prediction margin (correct-versus-incorrect gap) of the top 5 percent of concepts. Robustness of core knowledge	Maintaining 1.0 is the goal
Operation-plane cohesion	Mean pairwise cosine of the operation-plane (reasoning-axis) embeddings. The same computation as same-kind cohesion, applied to the operation plane	Lower keeps operators separated

The frozen model's starting values are holdout 5.006, same-kind cohesion 0.127, cross-kind separation 0.031, core strength 1.0, and operation-plane cohesion 0.081. The environment is in Appendix B.

5. Results

5.1 Simultaneous Compression and Grammar Preservation

When rumination is applied only to the data-plane embeddings, the grammar holdout barely changes while same-kind cohesion rises.

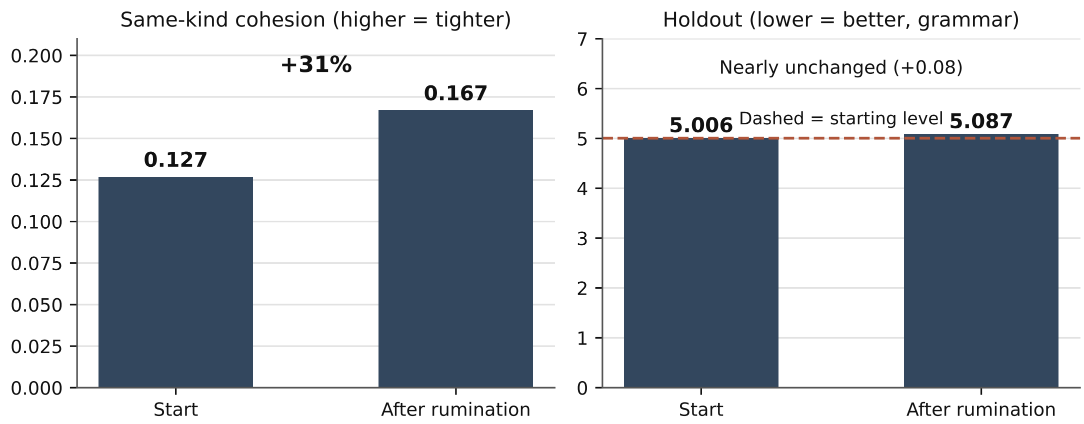
Measured

Measuring the same-kind rumination experiment (pull strength 0.05, 5 probe groups, renormalization on), over 800 steps:

- Same-kind cohesion 0.127 ▶ 0.167 (about +31 percent tighter)
- Grammar holdout 5.006 ▶ 5.087 (change +0.08, essentially preserved)
- Cross-kind separation maintained at 0.031 , core strength maintained at 1.0

Because only the data plane was pulled while the three parts responsible for grammar remained frozen, the ability to speak does not collapse even as the concepts tighten.

Figure 5-1. Compression and grammar preservation



Rumination tightens same-kind cohesion by 31 percent, from 0.127 to 0.167 (left), while the grammar holdout barely changes, from 5.006 to 5.087 (right). Concepts are organized while the ability to speak is protected.

5.2 Trade-off Between Cohesion and Holdout

Sweeping the pull strength α and the decay rate λ over a grid reveals a trade-off: stronger pull tightens further but worsens the grammar holdout.

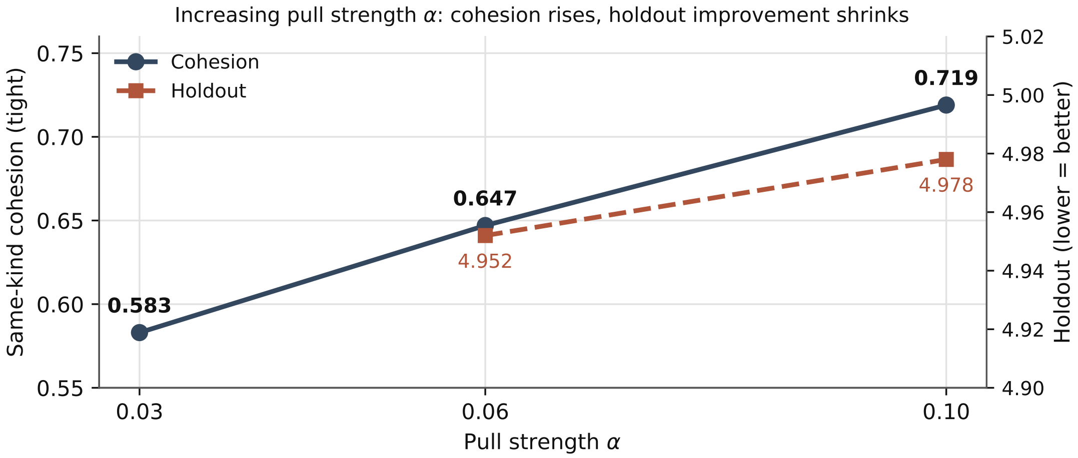
Measured

Measurements from the grid search (decay rates 0.003 and 0.006; pull strengths 0.03, 0.06, and 0.1; each combination measured). Same-kind cohesion and best holdout by pull strength (at decay rate 0.006):

Pull strength α	**Same-kind cohesion**	**Best holdout (change)**
0.03	0.583	4.917 (−0.09)
0.06	0.647	4.952 (−0.05)
0.10	0.719	4.978 (−0.03)

Stronger pull raises cohesion (from 0.583 to 0.719) but the holdout improvement shrinks. Pushing cohesion alone gradually costs grammar. The balance point between the two metrics lies at an intermediate strength.

Figure 5-2. Cohesion and holdout versus pull strength



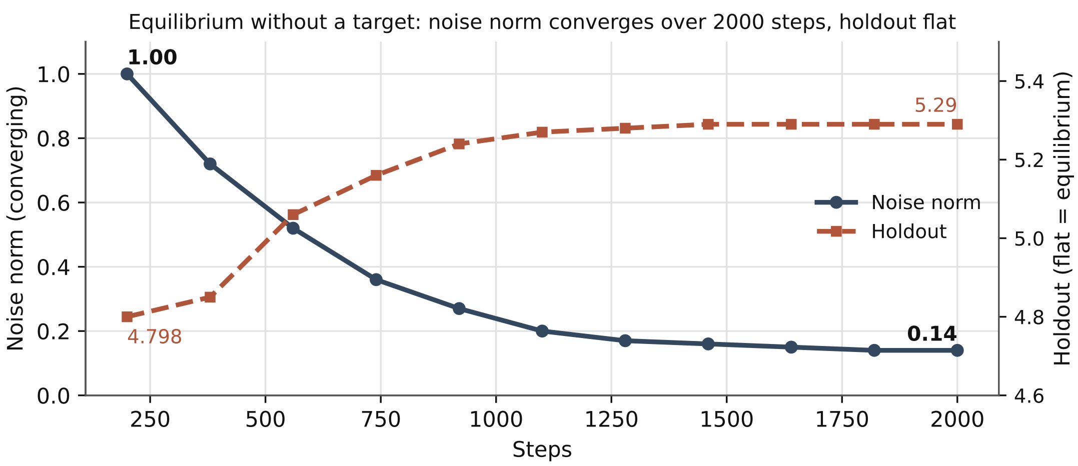
As the pull strength α grows, same-kind cohesion (solid line) rises from 0.583 to 0.719, but the holdout improvement (dashed line, lower is better) dulls. Tightness and grammar preservation pull against each other in a trade-off whose balance point lies in the middle.

5.3 Equilibrium Without a Target Value

Running with only pull and decay for a long time converges to an equilibrium even though no set point is given.

> **Measured**
>
> Measuring the always-on experiment for 2000 steps: the holdout flattens from an early 4.798 to around 5.29 later, the noise norm (deviation from the original) converges from 1.0 to 0.14, and same-kind cohesion 0.167 and core strength 1.0 are maintained. There is no divergence.

Figure 5-3. Equilibrium convergence in the long run



Over 2000 steps the noise norm (solid line) converges from 1.0 to 0.14 and the holdout (dashed line) flattens. Even with no target value given, the system settles on its own at the equilibrium where pull and decay tie, and does not diverge.

Reproduce

The figures in Sections 5.1 through 5.3 are reproduced with the following single line after completing the preparation in Appendix C.

```
cd probe && python3 paper2_rumination_probe.py
```

In the output, same-kind cohesion 0.127 to 0.167, holdout 5.006 to 5.087, and equilibrium noise norm converging from 1.0 to around 0.14 correspond to the figures in the text. The noise norm fluctuates by about 0.01 across runs due to the nondeterminism of GPU atomic additions.

5.4 Downstream Discrimination Before and After Rumination

The holdout is next-character prediction difficulty and thus checks only grammar preservation. Whether rumination also preserves downstream tasks such as answer selection is measured by 2AFC (two-alternative forced choice) discrimination. For 16 elementary-knowledge questions, a correct word and a plausible wrong word are placed as candidates, and a question counts as correct when the log probability at the answer position is higher for the correct word. Chance level is 50%, and the prior-preference baseline—comparing the answer words alone with the question removed—is 9/16.

There are two runs. The first run applied the published schedule of Section 5.1 as-is for 800 steps. However, since this schedule draws seeds from the pool of certain concepts and barely moves the question-answer candidate words (tokens actually moved in 6/16 questions), the before-after equality in this run is substantively verified only on those 6 questions. The second run is a stress run that forcibly injects the candidate words themselves as seeds. It makes semantically neighboring candidates such as “hot” and “cold” actually cluster (minimum cosine of question tokens against the original 0.233, tokens moved in 11/16 questions), directly measuring whether discrimination holds even when the candidate embeddings are strongly reoriented.

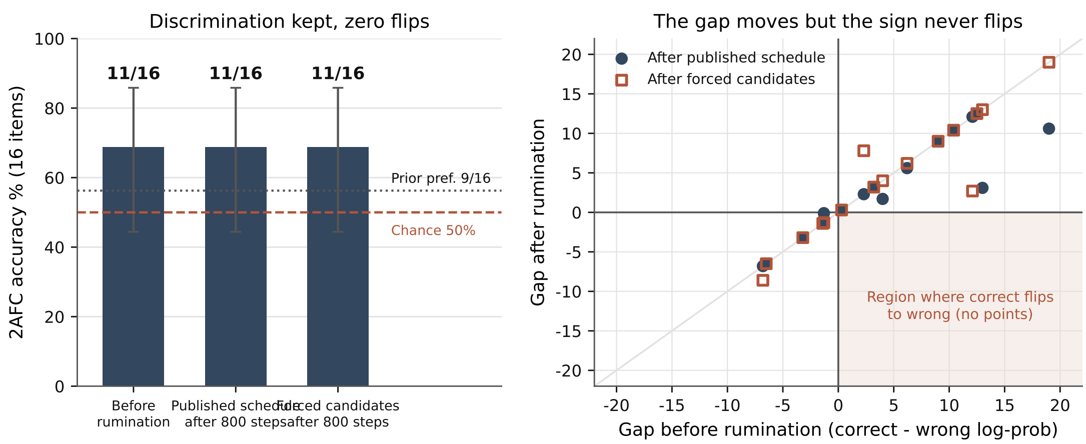
Measured

Discrimination is maintained in both runs.

Condition	2AFC accuracy	Flips (correct to wrong, wrong to correct)	Questions moved	Holdout
Before rumination	11/16 (confidence interval 44–86%)	-	-	5.006
After 800 steps, published schedule	11/16	0, 0	6/16	5.087
After 800 steps, stress run	11/16	0, 0	11/16	5.113

Per-question margins (log-probability gap between correct and wrong answers) move by nats (+19.0 to +10.6, +13.0 to +3.1, etc.), but not a single question crosses sign. The mean log probability of correct answers holds from -6.76 to -6.71 under the published schedule and declines gently to -7.58 under the stress run.

Figure 5-4. Downstream 2AFC discrimination before and after rumination



Left: 2AFC accuracy at three time points, with error bars showing Wilson 95% confidence intervals. Even in the stress run that forcibly clusters the candidate words, 11/16 is maintained with no flips. Right: before-after scatter of per-question margins; the margins move, but no point falls in the quadrant where a correct answer flips to wrong.

We state the interpretive limits of this measurement. With only 16 questions, the confidence interval for accuracy 11/16 is 44 to 86%, which includes chance at 50%, and the observation of zero flips rules out degradation only up to about 17 percentage points (rule of three). Since the prior-preference baseline is 9/16, a substantial part of this metric reflects dialogue-frame preference for the answer words rather than question understanding. When an answer candidate is a single bundle token, its own embedding does not enter the score computation directly under the low-rank head structure, so the effect of rumination is transmitted only through the context-encoding path. This result is therefore not a proof of the general proposition that rumination preserves downstream ability, but an observation that on this frozen model and this schedule the discrimination sign holds even when candidate embeddings are strongly reoriented. Generative dialogue quality was not measured (Section 7).

Reproduce

The downstream discrimination of this subsection is reproduced with the following single line after completing the preparation in Appendix C.

```
cd probe && python3 paper2_rumination_qa_probe.py
```

In the output, the prior-preference baseline 9/16, 11/16 maintained with 0 flips in both the published-schedule and stress runs, and the stress run's minimum question-token cosine 0.233 correspond to the figures in the text.

6. Discovery of the Reasoning Axis and an Unfinished Experiment

This section reports an experiment that separates the axis solved by rumination from the axis that rumination alone does not organize and that requires a different mechanism. This is not a failure but a process of pinpointing the next direction.

6.1 The Reasoning Axis (Operation Plane) Does Not Move Under Feedback Rumination

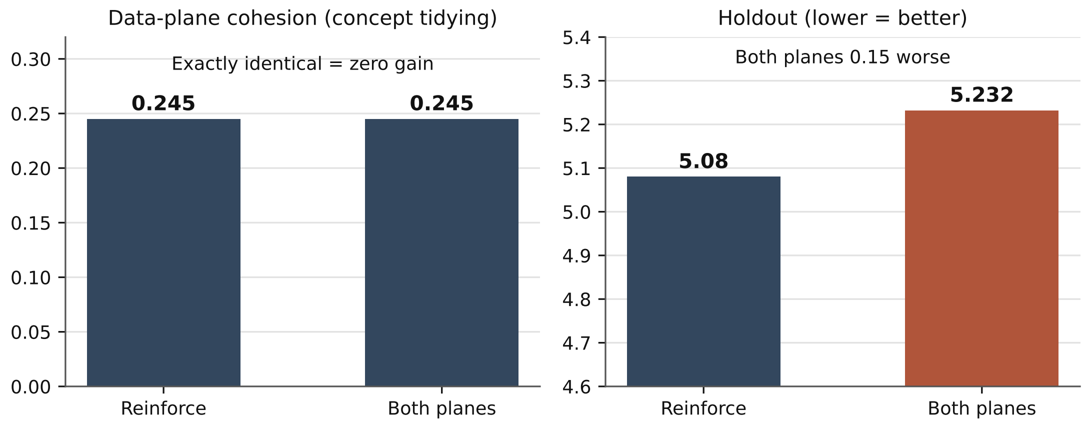
We compared, side by side, applying rumination to the data plane only (reinforce) and to the data plane and operation plane together (both-plane), to check whether feedback rumination also organizes the operation plane (the reasoning axis). Data-plane cohesion in the table below is the same metric as same-kind cohesion in Section 5, computed on the data plane; because this experiment differs in settings and steps, its values differ from those of Section 5.

Both-plane rumination comparison (600 steps, measured):

Condition	Data-plane cohesion	Operation-plane cohesion	Holdout
Reinforce (data plane only)	0.245	0.081	5.08
Both-plane (data + operation)	0.245	0.192	5.232

Even when the operation plane is also ruminated, data-plane cohesion is exactly the same at 0.245 . That is, the organization of concept similarity is already fully accomplished by data-plane rumination alone. That only operation-plane cohesion rises separately from 0.081 to 0.192 while the holdout worsens slightly means that the operation plane (the reasoning axis) is not organized by the similarity-pull loop that feedback is . Reasoning is an axis distinct from similarity and requires a loop other than feedback. This experiment is the unfinished experiment that pinpointed that fact, and the next direction.

Figure 6-1. The reasoning axis does not move under feedback rumination



Data-plane cohesion is exactly the same at 0.245 for reinforce and both-plane, showing that organizing concept similarity is sufficient with data-plane rumination alone (left). That only operation-plane cohesion rises separately while the holdout worsens slightly (right) pinpoints that the reasoning axis is not organized by feedback rumination and requires a different loop.

6.2 Similarity by Rumination; Reasoning and Its Integration by a Different Loop

In summary, feedback rumination organizes concept similarity on the data plane. That is what this paper has established by measurement. In contrast, reasoning (the operation-plane axis), and the self-rumination that mixes similarity consolidation with reasoning to run on its own without input, require a loop other than feedback. Feedback rumination is a geometric organization by centroid pull, so it suits gathering similarity, but reasoning has the character of a traversal that follows and combines concepts (e.g., depth-first search) and is not organized by the same pull loop. Put differently, the organizing principles of the two planes are opposite. The pull and decay of rumination are a device that refines a static embedding table by circulation (embeddings flowing around, like the convection of heated gas), which suits the data plane that carries content. An operator, however, functions only by not overlapping with other operators. In the measurements of Part 3, 72.5 percent of distinct operation pairs are nearly orthogonal and same-direction pairs are 0 percent, and what organized the operation plane was not resemblance but the prediction credit passing through the gate. Refining operators by pulling makes them gather and overlap, so they cease to be operations. The data plane gathers by resemblance and is refined; the operation plane separates by use and becomes sharper. This integration is an unfinished topic to be covered in the final part of this series; at present, external input (a person or new data) plays the role of the rumination source for creation. This is therefore not a defect of rumination but the boundary of its domain and the starting point of the next part.

7. Limitations and Next Measurement Tasks

The baseline of this paper is a reference class of one-person development, a self-built exact-adjoint backpropagation engine (hand-derived without automatic differentiation), and a single consumer graphics processing unit. The items below are not defects but next measurement tasks that build on strengths already measured in this class.

- Practical dialogue value : Section 5.4 showed that two-candidate discrimination is maintained before and after rumination, but a side-by-side comparison of generative dialogue quality is a next measurement task. With only 16 discrimination questions, an equivalence claim also requires a larger question bank.
- Comparison baselines : A side-by-side comparison with gradient-based self-reuse such as born-again networks or self-distillation is a next measurement task.
- Scale : Confirming equilibrium behavior on larger models and longer runs is a next measurement task.
- Background theory : The self-referential theory [1] is the motivation for this paper, not its evidence; this paper claims only measured facts.

8. Conclusion

This paper presented rumination learning, which reorganizes concept embeddings with forward-pass arithmetic alone—no gradient, no backpropagation, no external data. By measurement, rumination makes same-kind concepts 31 percent tighter while preserving grammar, converges to an equilibrium without a target value, and maintains downstream two-candidate discrimination without flips even in a stress run that strongly reoriented the candidate embeddings. It also pinpointed that the operation plane (the reasoning axis) is not organized by feedback rumination, so reasoning requires a different loop. The role of rumination is the organization of data similarity; reasoning and its integration belong to a different mechanism. Rumination reorganizes concepts using only internal material without outside data; fed into the training loop it serves as retraining, and during dialogue it serves as a deep-thinking skill. Where the engine of Part 1 computed exact gradients without automatic differentiation, this paper is self-organization that uses no gradient at all. The two coexist within one codebase as different learning principles. Why rumination is designed to run with no outside input at all is a question this paper does not answer; the answer is deferred to the final part of the series.

9. Series Preview

Series preview

- Part 3 — Orthogonal data-plane and operation-plane tokens, and the convolutional scale structure : Addresses the orthogonal structure of the operation plane and data plane that this paper froze, and analyzes head-on the depth and scale combinations of circulant-matrix channel mixing and filter-bank channel mixing.

- Part 4 — World-first developmental curriculum learning : Covers how the frozen model behind this paper was trained: developmental-order learning that forms sensory, spatial, and logical axes before language.
- Part 5 — Cache-dictionary tokenization and dynamic embedding : Covers the vocabulary layer on which concept embeddings sit, as a method that makes vocabulary expansion nearly free through a two-layer structure of syllable atoms and a bundle dictionary.
- Part 6 — Control signals emerging from self-distribution and geometry monitoring : Describes, using only measured statistics, the signals by which the model reads its own output distribution and embedding geometry to route certain, vague, and unknown states.
- Final part (unfinished) — Integrating similarity consolidation and reasoning traversal : Aims at self-rumination that runs on its own without input by weaving together data-plane similarity consolidation (this paper) and operation-plane reasoning traversal. The final assembly is not complete, so it is not yet written; it will be treated separately once measurements stand.

References

[1] Han, H. (2026). Banya Framework: An Axiom-Based Science Mining Engine for Deriving Fundamental Physical Constants. Zenodo. https://doi.org/10.5281/zenodo.20301304 . (Background natural philosophy of this series; prior axiom document)

[2] Han, H. (2026). A Language-Model Training Engine Using Hand-Derived Closed-Form Exact Adjoints Without Automatic Differentiation. Banya Research Series, Part 1. Zenodo. https://doi.org/10.5281/zenodo.21385447

[3] Hebb, D. O. (1949). The Organization of Behavior. Wiley.

[4] Hinton, G., Vinyals, O., Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.

[5] Furlanello, T., Lipton, Z., Tschannen, M., Itti, L., Anandkumar, A. (2018). Born-Again Neural Networks. ICML. arXiv:1805.04770.

[6] Papyan, V., Han, X. Y., Donoho, D. L. (2020). Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training. PNAS 117(40):24652-24663.

Appendix A. Rumination Constants

Table A-1. Rumination learning constants

Constant	Value	Meaning
Pull strength α	0.05	Rate of pulling toward the centroid
Decay rate λ	0.012	Rate of restoring toward the original
Ruminations per step	16	Number of seeds drawn per step
Budget decay ρ	0.8	Budget multiplied at each depth
Leaf threshold τ	0.14	Boundary beyond which no further expansion occurs
Same-kind group maximum	60	Upper bound of concepts one rumination gathers
Child width	3	Number of cosine-nearest children per node

Appendix B. Measurement Environment

Table B-1. Computer specifications used for measurement

Item	Specification
Graphics processing unit (GPU)	NVIDIA GeForce RTX 3090 Ti (24 GB)
Operating system	Ubuntu 24.04.4 LTS
Software	Python 3.12.3, numpy, cupy 14.1.1, CUDA 13.3.1
Learning background	Rumination applied only to the data-plane embeddings of an elementary-stage frozen model (about 190,000 steps) trained with the developmental curriculum

The measured figures in Sections 5 and 6 were obtained in the environment above.

Appendix C. Reproduction Guide

The measured figures in this paper are reproduced with the public reproduction package. Follow the five steps below in order.

1. Requirements. An NVIDIA graphics processing unit (GPU) with CUDA and Python 3.12 are required. Install the Python packages in the unpacked folder with the following. For cupy, use the distribution matching the installed CUDA version (e.g., cupy-cuda13x for CUDA 13).

```
pip install -r requirements.txt
```

2. Get the code. Download and unpack the zip bundle. To get it as a repository, use the github.com/ubmscoin/banya/banya_ai_paper_code folder.

```
wget https:
//ubmscoin.github.io/banya/banya_ai_paper_code/banya_ai_paper_code.zip
```

```
unzip banya_ai_paper_code.zip && cd banya_ai_paper_code
```

3. Get the models. The three frozen models (471 MB total) are on Zenodo at doi.org/10.5281/zenodo.21383724. The following single line performs both the download and the integrity check (sha256, file fingerprint comparison). To do it by hand, download the three files from the record above, place them in the model folder, and verify with `sha256sum -c checksums.sha256`.

```
cd model && bash download.sh && cd ..
```

4. Build the corpus. The corpus is produced by generators without distributing source text. The following single line generates all 23 corpora into the banya_world_data folder.

```
cd corpus_build && bash build_all.sh && cd ..
```

5. Reproduce the measurements. The probes in the probe folder reproduce the figures of each section. The figures of this paper are covered by the probes below. Target values are printed alongside the output, so they can be checked directly against the text.

Probe	**Figures reproduced**	**Output to check**
paper2_rumination_probe.py	5.1 compression and preservation, 5.3 equilibrium	Cohesion 0.127 to 0.167, holdout 5.006 to 5.087, noise norm from 1.0 to around 0.14
paper2_rumination_qa_probe.py	5.4 downstream discrimination	Prior preference 9/16, 11/16 maintained in both runs with 0 flips, stress-run minimum cosine 0.233

The package's folder layout and per-file descriptions are on the index page.

Banya Research Series, Part 2 (rumination). Gradient-free self-organization. The quantitative figures were measured directly in this paper under the environment of Appendix B. The background natural philosophy is cited as the prior axiom document [1] but is not the evidence for this paper's claims; this paper claims only measured facts. Part 1 (the engine) is cited. Orthogonal tokens and convolution, the developmental curriculum, cache-dictionary tokenization, and emergent control signals are covered in Parts 3 through 6, respectively.